SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai 2026

ASSESSMENT TOOL -3

CO2_AT3

Name : N.Sunil Kumar

Register No : 192472330

Course code : CSA6502

Course name: Generative Ai And Large Scale Models

Faculty name: Dr.Nithisha

QUESTION :-

Domain-Specific Prompting vs Fine-Tuning

A legal company wants its LLM to consistently produce responses using legal terminology and a specific document format.

Task: Determine whether prompt engineering, few-shot prompting, fine-tuning, or PEFT would be most appropriate. Justify your decision based on the scenario.

ANSWER :-

1. Introduction

Artificial Intelligence (AI) has significantly transformed the way businesses process information and automate repetitive tasks. One of the most important developments in AI is the emergence of Large Language Models (LLMs) such as GPT, LLaMA, Claude, Gemini, and Mistral. These models are trained on massive amounts of text data, allowing them to understand natural language, answer questions, summarize documents, generate code, translate languages, and create high-quality written content.

In the legal industry, documentation plays a vital role because every legal process requires accurate, structured, and professional documents. Legal organizations prepare a wide range of documents, including contracts, employment agreements, lease agreements, non-disclosure agreements (NDAs), legal notices, affidavits, compliance reports, and case summaries. Traditionally, preparing these documents requires experienced legal professionals, consumes considerable time, and demands careful attention to legal terminology and formatting.

Large Language Models offer an opportunity to automate legal document generation. However, a general-purpose LLM may not always produce responses that satisfy legal standards. The model may use informal language, omit essential clauses, or generate inconsistent document structures. Since legal documents require precision, consistency, and compliance with legal regulations, organizations must adapt LLMs specifically for legal applications.

There are several approaches available for adapting LLMs to specialized domains. Prompt Engineering guides the model using carefully designed instructions without modifying the model itself. Few-Shot Prompting improves the output by providing example documents within the prompt.

2. Problem Statement

A legal company intends to implement an Artificial Intelligence system capable of automatically generating professional legal documents. The organization handles a large number of legal cases every day, requiring the preparation of contracts, legal notices, employment agreements, lease agreements, confidentiality agreements, compliance reports, and legal summaries.

Although a general-purpose Large Language Model can generate text, it does not consistently follow legal terminology or company-specific document templates. The generated documents often require manual corrections because important legal clauses may be missing, formatting may differ across documents, and terminology may not match the organization's standards.

These inconsistencies increase the workload of legal professionals and reduce the efficiency of document preparation. Manual review becomes necessary before every document can be approved, consuming valuable time and increasing operational costs.

To overcome these challenges, the company must determine the most appropriate method for adapting the LLM. The available options include Prompt Engineering, Few-Shot Prompting, Fine-Tuning, and Parameter-Efficient Fine-Tuning (PEFT). The selected approach should produce consistent legal terminology, standardized formatting, high-quality document generation, and reliable performance across different legal document types.

4. Objectives

The primary objective of this project is to identify the most suitable approach for adapting a Large Language Model to generate professional legal documents that meet organizational standards.

The specific objectives are:

Objective 1: Generate Consistent Legal Terminology

The system should consistently use formal legal language instead of informal or conversational text. Every generated document should include legally accepted terms, clauses, and expressions commonly used by legal professionals.

Objective 2: Maintain Standardized Document Structure

Every legal document should follow a predefined template. Sections such as the title, parties involved, definitions, obligations, confidentiality, governing law, dispute resolution, and signatures should always appear in the correct order.

Objective 3: Improve Document Accuracy

The adapted LLM should minimize grammatical mistakes, missing clauses, and inconsistent legal wording. Higher accuracy reduces the need for manual corrections and improves overall document quality.

Objective 4: Reduce Human Effort

By automating document generation, legal professionals can spend more time reviewing complex legal issues instead of drafting repetitive documents from scratch.

Objective 5: Improve Organizational Productivity

Faster document generation enables organizations to handle more legal cases within shorter timeframes, increasing productivity and reducing operational costs.

Objective 6: Ensure Scalability

The solution should support multiple document types and be capable of adapting to future legal requirements without significant redevelopment.

5. Existing Approaches for Domain Adaptation

Methods used to adapt LLMs for specific domains.

5.1 Prompt Engineering
Uses clear instructions to guide the model without retraining.

5.2 Few-Shot Prompting
Provides a few examples to improve response quality.

5.3 Fine-Tuning
Retrains the model using domain-specific data.

5.4 Parameter-Efficient Fine-Tuning (PEFT)
Updates only a small number of parameters (e.g., LoRA, Adapters) to reduce training cost.

6. Prompt Engineering

Introduction

Prompt Engineering is one of the simplest and most commonly used techniques for interacting with Large Language Models (LLMs). It involves designing clear, specific, and structured prompts that instruct the model to generate the desired output. Instead of modifying or retraining the model, Prompt Engineering relies on the wording and quality of the input prompt to influence the generated response.

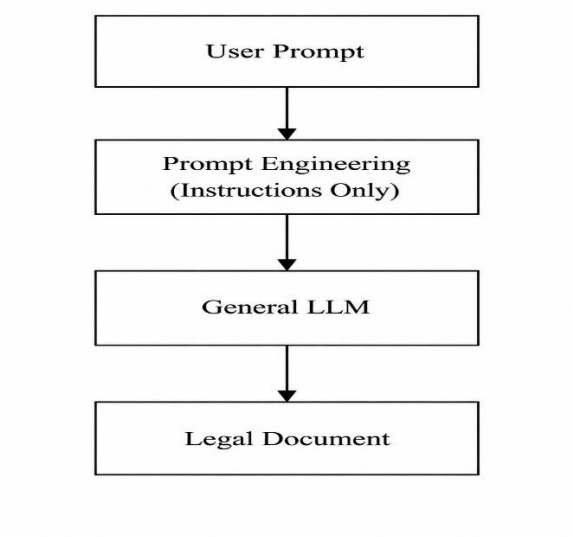
In a legal company, Prompt Engineering can be used to generate legal documents by providing detailed instructions. For example, the prompt may specify the type of legal document, required sections, legal writing style, and formatting rules. If the prompt is well designed, the LLM can produce high-quality legal documents that follow the requested structure.

Working Process

The Prompt Engineering process consists of the following steps:

1. The user writes a detailed prompt describing the legal document.
2. The LLM analyzes the instructions provided in the prompt.
3. Based on its pre-trained knowledge, the model generates the legal document.
4. The generated document is reviewed by the user.
5. If the result is unsatisfactory, the prompt is modified and the process is repeated.

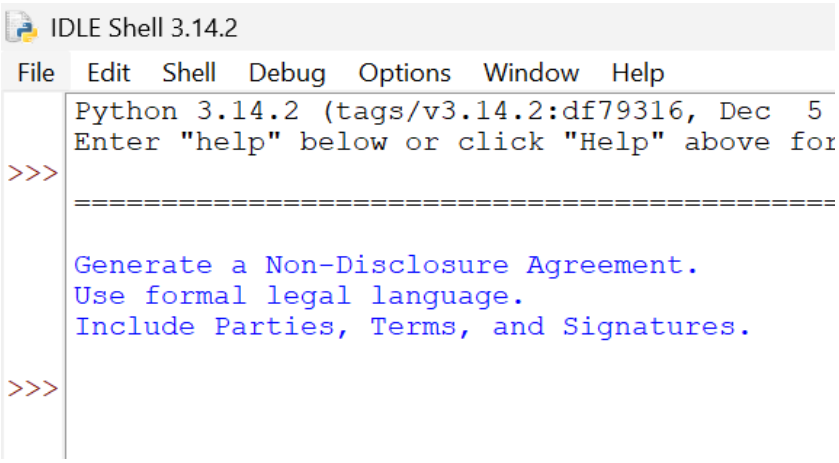
Prompt Engineering Architecture



PYTHON CODE:-

```
# Prompt Engineering

prompt = """

Generate a Non-Disclosure Agreement.

Use formal legal language.

Include Parties, Terms, and Signatures.

"""

print(prompt)
```

SAMPLE OUTPUT :-



Advantages

- Simple and easy to implement.
- No model retraining is required.
- Low computational cost.
- Suitable for quick experiments.

Limitations

- Output quality depends heavily on prompt design.
- Responses may vary for similar prompts.
- Does not permanently learn legal terminology.

7. Few-Shot Prompting

Introduction

Few-Shot Prompting is an extension of Prompt Engineering in which the model is provided with a small number of example inputs and outputs before performing a new task. These examples help the model understand the expected writing style, formatting, and terminology. Instead of learning through training, the model temporarily uses the examples as guidance while generating the response.

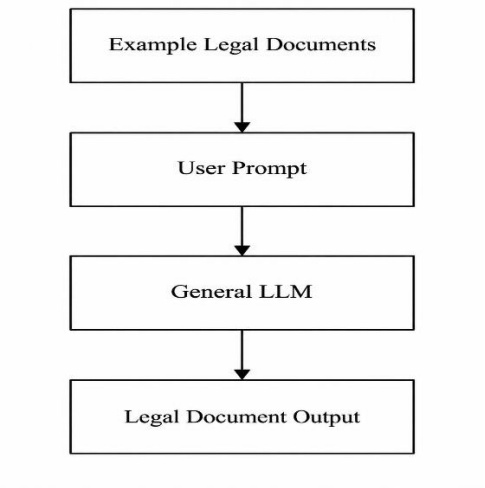
In legal document generation, a Few-Shot Prompt may contain examples of employment agreements, lease agreements, or non-disclosure agreements before asking the model to create a new contract. By observing these examples, the LLM is more likely to generate documents with similar formatting and legal language.

Although Few-Shot Prompting improves consistency compared to basic Prompt Engineering, the examples must be included every time the model is used.

Working Process

1. Prepare two or more sample legal documents.
2. Include the examples in the prompt.
3. Add the new document request.
4. The LLM analyzes the examples and identifies common patterns.
5. The model generates a new legal document following the same structure.

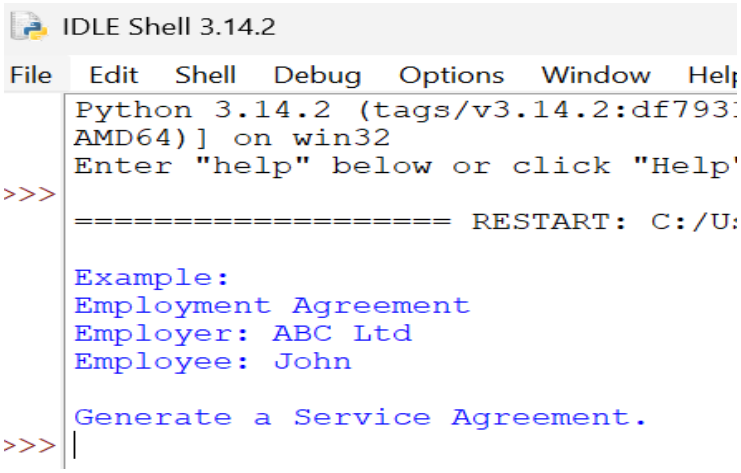
Few-Shot Prompting Architecture



PYTHON CODE:-

```
# Few-Shot Prompting
example = """
Example:
Employment Agreement
Employer: ABC Ltd
Employee: John
"""
query = "Generate a Service Agreement."
print(example)
print(query)
```

SAMPLE OUTPUT :-



Advantages

- Better consistency than Prompt Engineering.
- No additional model training.
- Learns formatting from examples.
- Easy to implement.

Limitations

- Long prompts consume more tokens.
- Examples must be provided every time.
- The model does not permanently learn legal knowledge.

8. Fine-Tuning

Introduction

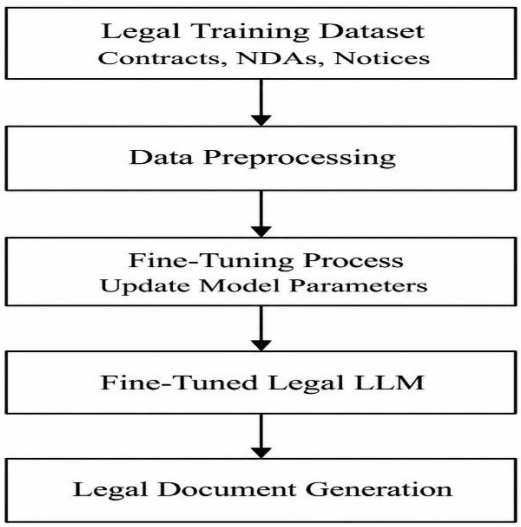
Fine-Tuning is the process of retraining a pre-trained Large Language Model using domain-specific data. Instead of depending only on prompts, the model learns directly from thousands of legal documents such as contracts, employment agreements, lease agreements, legal notices, and court documents. During training, the model updates its internal parameters, allowing it to permanently learn legal terminology, writing style, and document structure.

Fine-Tuning is considered the most effective solution for organizations that require high consistency and accuracy. Once the model has been fine-tuned, users only need to provide simple instructions, and the model automatically generates professional legal documents that follow organizational standards.

Working Process

1. Collect a large dataset of legal documents.
2. Clean and preprocess the dataset.
3. Convert the documents into tokens.
4. Train the model using the legal dataset.
5. Evaluate the model's performance.
6. Deploy the fine-tuned model for legal document generation.

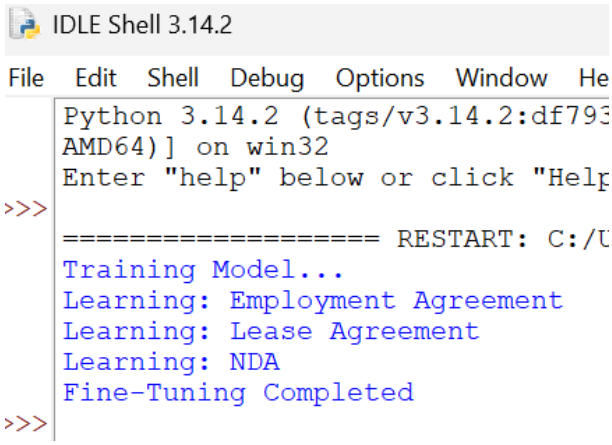
Fine-Tuning Architecture



PYTHON CODE:-

```
# Fine-Tuning
documents = [
    "Employment Agreement",
    "Lease Agreement",
    "NDA"]
print("Training Model...")
for doc in documents:
    print("Learning:", doc)
print("Fine-Tuning Completed")
```

SAMPLE OUTPUT :-



Advantages

- High accuracy.
- Consistent legal terminology.
- Standardized document formatting.
- Reduced manual editing.
- Suitable for large organizations.

Limitations

- Requires large training datasets.
- High computational cost.
- Training takes considerable time.

9. Parameter-Efficient Fine-Tuning (PEFT)

Introduction

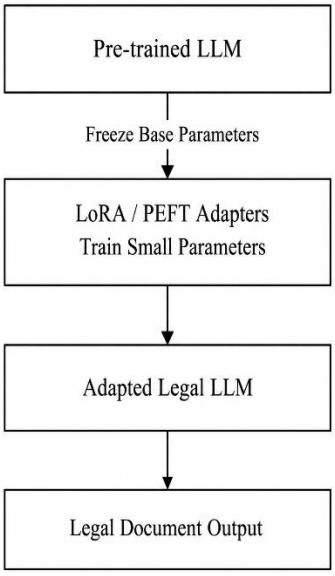
Parameter-Efficient Fine-Tuning (PEFT) is a modern technique that adapts a Large Language Model by updating only a small portion of its parameters instead of retraining the entire model. The original model remains unchanged, while additional lightweight components, known as adapters, are trained for the specific domain.

One of the most popular PEFT methods is LoRA (Low-Rank Adaptation). LoRA inserts small trainable matrices into selected layers of the model, significantly reducing the number of parameters that need to be updated. This approach reduces GPU memory usage, training time, and storage requirements while maintaining performance close to full Fine-Tuning.

Working Process

1. Load the pre-trained language model.
2. Freeze the original model parameters.
3. Insert LoRA or other PEFT adapters.
4. Train only the adapter parameters using legal documents.
5. Save the trained adapters.
6. Use the adapted model to generate legal documents.

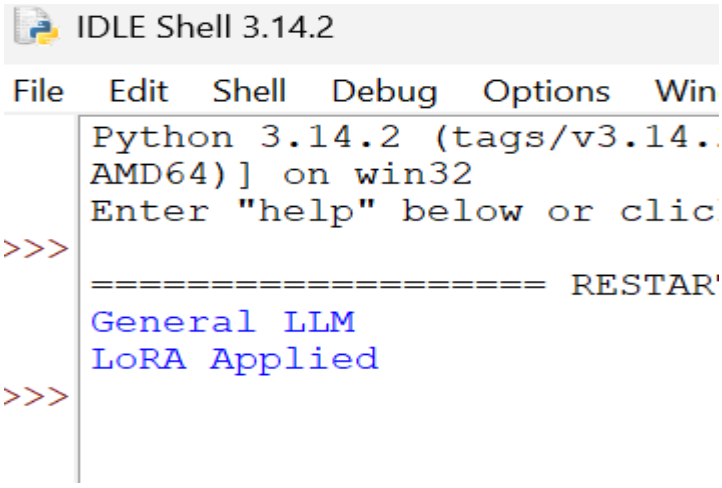
PEFT (LoRA) Architecture



PYTHON CODE:-

```
# PEFT (LoRA)

base_model = "General LLM"

adapter = "LoRA Applied"

print(base_model)

print(adapter)
```

SAMPLE OUTPUT:-



Advantages

- Lower training cost.
- Reduced GPU memory usage.
- Faster training.
- Easy deployment.
- Performance close to full Fine-Tuning.

Limitations

- Slightly less flexible than full Fine-Tuning.
- Adapter configuration requires expertise.
- Performance depends on the quality of training data.

10. Why Fine-Tuning is the Best Choice

For the given legal company scenario, Fine-Tuning is the most appropriate technique because the organization requires consistent legal terminology and a fixed document format across all generated documents. Unlike Prompt Engineering and Few-Shot Prompting, Fine-Tuning permanently teaches the model legal writing patterns, enabling it to generate accurate and standardized legal documents with minimal user input.

If the organization has limited computational resources, PEFT (LoRA) is an excellent alternative because it provides nearly the same performance as Fine-Tuning while reducing training cost and memory requirements. Thus, Fine-Tuning is recommended for large legal organizations, whereas PEFT is suitable for small and medium-sized organizations seeking an efficient and cost-effective solution.

Conclusion

Large Language Models (LLMs) have become powerful tools for generating domain-specific content, including legal documents. Different adaptation techniques such as Prompt Engineering, Few-Shot Prompting, Fine-Tuning, and Parameter-Efficient Fine-Tuning (PEFT) offer different levels of performance, cost, and complexity.

Prompt Engineering is simple and inexpensive, making it suitable for basic tasks, while Few-Shot Prompting improves output quality by providing example documents. However, both techniques rely on carefully designed prompts and do not permanently teach the model domain-specific knowledge.

For a legal company that requires consistent legal terminology, standardized document formatting, and high-quality legal document generation, Fine-Tuning is the most suitable approach because it permanently adapts the model using legal datasets. If computational resources or budget are limited, PEFT (LoRA) provides an efficient alternative by training only a small number of model parameters while achieving performance close to full Fine-Tuning.